

Informe de Pruebas Unitarias

Proyecto: Mi Cita

Módulos: Seguridad, Orquestación de Citas y Núcleo de Citas

Información General

Proyecto: Mi Cita

Arquitectura: Capas (Controller / Business / Data)

Lenguaje: C# (.NET)

Framework de pruebas: xUnit

Mocking: Moq

Fecha: 2025

1. Introducción

Este documento consolida y documenta de manera formal las **pruebas unitarias** implementadas sobre tres componentes críticos del proyecto **Mi Cita**, los cuales forman parte del núcleo funcional del sistema de agendamiento de citas médicas.

Las pruebas unitarias permiten validar el comportamiento de la lógica de negocio de forma **aislada**, reduciendo riesgos, previniendo errores tempranos y asegurando la estabilidad del sistema ante futuros cambios.

Los componentes evaluados son:

- **RolBusiness** – Gestión de roles y seguridad.
 - **AppointmentOrchestrator** – Orquestación de reservas en tiempo real.
 - **CitationCoreService** – Cálculo y disponibilidad de bloques horarios.
-

2. Objetivos

2.1 Objetivo General

Validar mediante pruebas unitarias el correcto funcionamiento de los servicios de negocio más críticos del sistema Mi Cita.

2.2 Objetivos Específicos

- Validar la gestión de roles del sistema.
- Verificar la lógica de bloqueo, liberación y confirmación de citas.
- Garantizar el cálculo correcto de bloques horarios disponibles.

- Asegurar el aislamiento de dependencias mediante mocks.
 - Documentar los resultados de forma clara y reproducible.
-

3. Alcance

Las pruebas cubren exclusivamente la **capa Business**, excluyendo:

- Base de datos real
 - SignalR real
 - Servicios de correo reales
 - Pruebas de integración o end-to-end
-

4. Herramientas Utilizadas

- .NET
 - C#
 - xUnit
 - Moq
 - Entity Framework Core (mock / in-memory)
 - SignalR (mock)
-

PRUEBAS UNITARIAS – RolBusiness

5. Componente Evaluado

Clase: RolBusiness

Responsabilidad: Gestión de roles y permisos del sistema.

6. Casos de Prueba Implementados

Obtener todos los roles

```
[Fact]
public async Task GetAll_ShouldReturnListOfRoles()
{
    var roles = new List<Rol>
    {
        new Rol { Id = 1, Name = "Admin" },
        new Rol { Id = 2, Name = "User" }
    };
}
```

```

_mockRolData.Setup(r => r.GetAll()).ReturnsAsync(roles);

var result = await _rolBusiness.GetAll();

Assert.NotNull(result);
Assert.Equal(2, result.Count());
}

```

Crear un rol

```

[Fact]
public async Task Save_ShouldReturnCreatedRole()
{
    var rolDto = new RolCreatedDto { Name = "NewRole" };
    var rolEntity = new Rol { Id = 10, Name = "NewRole" };

    _mockRolData.Setup(r => r.Save(It.IsAny<Rol>())).ReturnsAsync(rolEntity);

    var result = await _rolBusiness.Save(rolDto);

    Assert.NotNull(result);
    Assert.Equal(10, result.Id);
}

```

Obtener rol por ID

```

[Fact]
public async Task GetById_ShouldReturnRole_WhenExists()
{
    var rolEntity = new Rol { Id = 5, Name = "Manager" };
    _mockRolData.Setup(r => r.GetById(5)).ReturnsAsync(rolEntity);

    var result = await _rolBusiness.GetById(5);

    Assert.NotNull(result);
}

```

Eliminar rol

```

[Fact]
public async Task Delete_ShouldReturnTrue_WhenRoleExists()
{
    _mockRolData.Setup(r => r.Delete(3)).ReturnsAsync(true);

    var result = await _rolBusiness.Delete(3);
}

```

```

    Assert.True(result);
}

```

PRUEBAS UNITARIAS – AppointmentOrchestrator

7. Componente Evaluado

Clase: AppointmentOrchestrator

Responsabilidad: Controlar concurrencia, confirmación y notificación de citas.

8. Dependencias Mockeadas

```

Mock<ISlotLockStore>
Mock<ICitationsData>
Mock<IAppointmentNotifier>
Mock<IUserData>
Mock<INotificationBusiness>
Mock<INotificationSender>
Mock<ICitationNotificationService>

```

9. Casos de Prueba

Bloqueo de slot

```

[Fact]
public async Task TryLockAsync_ShouldLockSlot_WhenAvailable()
{
    var request = new SlotLockRequest
    {
        ScheduleHourId = 1,
        Date = DateTime.Today,
        TimeBlock = "08:00-09:00"
    };

    _lockStore.Setup(l => l.TryAcquireAsync(
        It.IsAny<SlotKey>(), "user1", It.IsAny<TimeSpan>(), It.IsAny<CancellationToken>()))
        .ReturnsAsync((true, DateTime.UtcNow.AddMinutes(2), "user1"));

    var result = await _orchestrator.TryLockAsync(request, "user1", CancellationToken.None);
}

```

```

        Assert.True(result.Success);
    }

```

Confirmación de cita

```

[Fact]
public async Task ConfirmAsync_ShouldCreateCitation_WhenSlotOwned()
{
    var slot = new SlotKey(1, DateTime.Today, "08:00-09:00");

    _lockStore.Setup(l => l.CheckAsync(slot, "1", It.IsAny<CancellationTok>()))
        .ReturnsAsync((true, DateTime.UtcNow, "1"));

    var result = await _orchestrator.ConfirmAsync(slot, 1, null, CancellationToken.None);

    Assert.True(result.success);
    Assert.NotNull(result.citationId);
}

```

PRUEBAS UNITARIAS – CitationCoreService

10. Componente Evaluado

Clase: CitationCoreService

Responsabilidad: Calcular bloques horarios disponibles para citas.

11. Casos de Prueba

Calcular bloques horarios

```

[Fact]
public void CalcularBloquesHorarios_ShouldReturnBlocks()
{
    var scheduleHour = new ScheduleHour
    {
        StartTime = new TimeSpan(8, 0, 0),
        EndTime = new TimeSpan(12, 0, 0),
        Shedule = new Shedule { NumberCitation = 4 }
    };

    var result = _service.CalcularBloquesHorarios(scheduleHour);
}

```

```

        Assert.Equal(4, result.Count);
    }

```

Filtrar bloques disponibles

```

[Fact]
public void FiltrarBloquesDisponibles_ShouldExcludeOccupied()
{
    var bloques = new List<TimeSpan>
    {
        new TimeSpan(8,0,0),
        new TimeSpan(9,0,0)
    };

    var ocupados = new List<TimeSpan> { new TimeSpan(9,0,0) };

    var result = _service.FiltrarBloquesDisponibles(bloques, ocupados);

    Assert.Single(result);
    Assert.True(result.First().EstaDisponible);
}

```

12. Resultados Generales

Componente	Estado
RolBusiness	Aprobado
AppointmentOrchestrator	Aprobado
CitationCoreService	Aprobado

13. Conclusiones

Las pruebas unitarias implementadas sobre estos tres componentes validan correctamente la lógica crítica del sistema **Mi Cita**, asegurando:

- Correcta gestión de roles.
- Control de concurrencia en reservas.
- Cálculo confiable de disponibilidad horaria.

Este enfoque mejora la calidad del software, facilita el mantenimiento y reduce riesgos en producción.

Fin del documento